MASTERMIND

REGOLE:

Lo scopo del giocatore è indovinare il codice segreto generato dal server nel minor numero di tentativi possibili. Il codice segreto è sempre composto da 4 pioli colorati (rossi, blu, verdi e gialli). Il giocatore ha un massimo di 10 tentativi per indovinare la combinazione segreta. Ad ogni tentativo il server valuta la combinazione proposta dal giocatore confrontandola con il codice segreto e assegna tanti pioli neri quanti sono i colori giusti nella posizione giusta e tanti pioli bianchi quanti sono, invece, i colori giusti ma nella posizione sbagliata. Nel conteggio dei pioli bianchi sono esclusi i colori già considerati dai pioli neri.

L'applicazione chiede al giocatore di autenticarsi e quindi di effettuare la registrazione o l'accesso per poter giocare. La registrazione, così come l'accesso, richiede al giocatore di inserire due credenziali testuali: username e password. Se l'username è già online, il server impedisce l'accesso, notificando che l'account è già in uso. Se l'accesso va a buon fine, viene aperto un menù di gioco in cui è possibile selezionare le seguenti modalità:

- Modalità Singolo (Giocatore contro Server): Il giocatore tenta di indovinare un codice generato casualmente dal server, inserendo le combinazioni tentativo fino a quando indovina il codice segreto o abbandona la partita selezionando 'esci'. Il client effettua i tentativi inviando i payload al server che computa i pioli e risponde con un pacchetto contenente il numero di pioli neri, bianchi e il flag booleano di vittoria.
- Modalità PvP (Multigiocatore 1vs1 a Turni): Consente a due utenti remoti di sfidarsi in tempo reale sulla stessa partita, a turni alternati, cercando di indovinare lo stesso codice segreto generato dal server. Il giocatore che indovina il codice segreto per primo vince la partita, guadagnando 10 punti nella classifica globale; il giocatore sconfitto, invece, subisce una penalità di 5 punti (il punteggio non può diventare negativo). E' presente una chat di gioco che permette ai giocatori di comunicare mandando un messaggio (broadcast) durante il proprio turno. Se un client si disconnette volontariamente o involontariamente (per esempio a causa di un crash di rete) durante un match 1vs1, il server assegna la vittoria a tavolino al giocatore rimasto, aggiornando i punti in classifica.
- Modalità Spettatore: L'utente può selezionare una partita in corso 1vs1 a cui assistere. Lo spettatore, quindi, osserva in tempo reale le combinazioni 'tentativo' effettuate dai giocatori e i conseguenti pioli neri e/o bianchi associati, oltre che i messaggi scambiati tra i 2 giocatori.
- Visualizzazione della Classifica: Permette all'utente di visualizzare i punteggi dei 5 migliori giocatori.

CRITTOGRAFIA:

La crittografia è stata implementata mediante un approccio a due livelli:

- Durante il trasferimento all'interno della rete, per evitare che i messaggi di gioco, le chat e le credenziali degli utenti possano essere intercettati attraverso attacchi di packet sniffing, i socket TCP standard sono stati incapsulati in un canale sicuro crittografato tramite il protocollo TLS (Transport Layer Security). Per cui se un malintenzionato intercetta i pacchetti di rete, vedrà solo una sequenza di caratteri incomprensibili e non il vero

contenuto del messaggio. Inoltre, tramite l'uso di certificati digitali, il protocollo garantisce che il server con cui sta parlando sia effettivamente chi dice di essere, prevenendo attacchi in cui un truffatore si finge il server legittimo. Ogni messaggio include un codice di autenticazione; questo assicura che i dati non vengano alterati o persi durante il tragitto. Se un byte viene modificato da una terza parte, la connessione si interrompe.

    - Quando il server accetta la connessione TCP (handshake TCP) sulla porta 12000, non comunica direttamente con il client, ma passa il socket alla funzione (avvolgi_server). Inizia quindi la procedura di handshake TLS, per cui il server genera una chiave privata (key.pem) che non verrà inviata al client ma servirà per decifrare i primi messaggi di scambio, mentre invia al client la chiave pubblica, nonché il proprio certificato digitale (cert.pem). Il client genera una chiave di sessione, la cifra utilizzando la chiave pubblica contenuta nel certificato e lo invia al server. Solo il server, quale unico possessore della corrispondente chiave privata (`key.pem`) può decifrare tale payload. A questo punto il client e il server usano la medesima Chiave di Sessione Simmetrica, per cui l'intera comunicazione viene cifrata utilizzando esclusivamente la chiave di sessione condivisa. Si passa, quindi, da una crittografia asimmetrica computazionalmente lenta ad una crittografia simmetrica più veloce e quindi adatta ad un ambiente di gioco.

- Al momento del salvataggio nel database locale: la sicurezza delle password memorizzate nel file di database è così gestita:
    - Generazione del Salt**:** All'atto della registrazione di un nuovo utente viene creata una stringa casuale univoca di 32 caratteri esadecimali (128 bit), usando la libreria secrets(). Se due utenti scelgono la stessa identica password, i loro hash finali sul database saranno differenti.
    - Funzione di Hashing**:** La password inserita dall'utente viene concatenata al salt generato. Questa stringa viene poi sottoposta a una funzione di hash unidirezionale non invertibile. Nel database JSON vengono salvati unicamente il salt e l'hash finale.
    - Verifica al Login**:** Quando un utente tenta di accedere, il server estrae dal database il salt specifico memorizzato per quell'username, lo concatena alla password digitata sul momento ed esegue lo stesso calcolo dell'hash. Se l'hash calcolato al momento combacia con l'hash memorizzato sul file,viene consentito l'accesso.

INTERFACCIA:

L'interfaccia è implementata nel modulo interfaccia.py utilizzando la libreria nativa Tkinter. L'applicazione è contenuta in una sola finestra e per passare da una schermata all'altra (per es: dal Menu alla Partita), l'interfaccia grafica adotta un approccio dinamico: ogni schermata è costruita inserendo i vari widget all'interno di un unico contenitore principale chiamato. Quando l'utente cambia modalità una funzione distrugge tutti i widget figli del contenitore, lasciando la finestra pulita e pronta per ospitare i widget della nuova schermata. Le schermate principali gestite sono quattro:

1. Menu Principale: presenta i 4 pulsanti per selezionare le modalità di gioco o consultare la classifica.
2. Tabellone di Gioco: contiene un oggetto per il disegno grafico della griglia di gioco e delle risposte del server, e un'area di testo a scorrimento per i log e la chat.

3. Classifica: una tabella formattata che mostra i top 5 giocatori ordinati per punteggio in ordine decrescente.
4. Tribuna Spettatori: mostra l'elenco delle partite in corso e un pulsante "Guarda" per poterle osservare.

L'utente interagisce con il gioco in maniera visivo, è l'interfaccia grafica si occupa di convertire le azioni in dati conformi al protocollo di rete:

- La scelta dei colori: quando l'utente clicca su uno dei 4 bottoni colorati in basso, quel colore viene convertito in una stringa numerica con numeri compresi fra `"1"` a `"4"` e lo inserisce nell'array.
- L'invio della giocata: cliccando su "Invia Mossa", il metodo invia_mossa() converte l'array numerico in un JSON con l'OpCode OP_MOSSA e lo invia direttamente sul socket TLS del server.

Se la chiamata bloccante socket.recv() venisse eseguita nel thread principale, la GUI smetterebbe di rispondere ai click e il sistema operativo la segnalerebbe come "Applicazione bloccata". Per risolvere questo problema, la GUI implementa il pattern Produttore-Consumatore mediato da una coda FIFO:

1. Il Produttore**:** sta in ascolto passivo sul socket. Quando riceve i byte JSON descritti nel protocollo del Canvas, li inserisce nella coda.
2. Il Consumatore**:** il main thread della GUI verifica periodicamente lo stato della coda senza sospendere il proprio flusso di elaborazione, garantendo la continuità dell'esecuzione. Se ci sono pacchetti, li estrae e aggiorna la grafica in totale sicurezza.

Questo disaccoppiamento garantisce che qualsiasi aggiornamento grafico avvenga sempre all'interno del thread principale, mantenendo fluida l'applicazione.

PROTOCOLLO:

Per governare la logica dell'applicazione, è stato definito un protocollo, basato sullo scambio di payload json. Per cui ogni pacchetto possiede un campo chiave op, che definisce la natura del messaggio (es. OP_LOGIN, OP_MOSSA, OP_TURNO). Il server associa gli OpCode a specifici metodi di gestione, mediante le dispatch tables. Questo approccio elimina gli if-else in cascata, rendendo il codice scalabile e più leggibile.L'utilizzo di JSON per l'incapsulamento dei dati e il protocollo permettono la facile integrazione di un client scritto in un linguaggio differente.

Vantaggi della Modularità:

Invece di delegare il routing dei messaggi a complesse e inefficienti catene di blocchi condizionali annidati (`if-elif-else`), il server utilizza dizionari Python per mappare direttamente gli OpCode ai riferimenti delle funzioni (es. `self.dispatch_pvp[tipo_ricevuto]`). Questo garantisce una risoluzione del metodo in tempo costante $O(1)$, ottimizzando le prestazioni del server sotto carico.

- Ogni metodo invocato dalla Dispatch Table ha un singolo compito, facilitando la lettura del codice, la ricerca di bug e il testing di moduli isolati.

- Se in futuro si volesse implementare una nuova funzionalità, non sarà necessario modificare la logica di routing esistente, bensì basterà definire un nuovo OpCode in protocollo_mm.py, scrivere il metodo dedicato in server.py e aggiungere una riga alla Dispatch Table corrispondente.
- L'utente viene processato esclusivamente dalla Dispatch Table associata al suo stato corrente (ad esempio: dispatch_autenticazione se non è loggato o dispatch_pvp se è in partita).

GESTIONE DELLA CONCORRENZA:

Il server adotta un modello di tipo multithreading, cioè in server.py il thread principale esegue un ciclo infinito in cui richiama esclusivamente il metodo bloccante (.accept()). Quando un client completa il three-way-handshake TCP e TLS, il server delega la gestione di quel socket ad un thread dedicato. In questo modo il thread principale torna istantaneamente in ascolto sulla porta 12000, mentre i thread figli gestiscono le sessioni in parallelo. Con l'uso dei thread, si introduce, però, il problema dell'accesso concorrente alle risorse. Tutti i thread figli, infatti, condividono lo stesso spazio di memoria globale per cui se due thread cercassero di aggiornare il punteggio sul file utenti.json nello stesso esatto millesimo di secondo, si verificherebbe una Race Condition che corromperebbe il file JSON. Per gestire la sincronizzazione dell'accesso alla memoria condivisa evitando race conditions adoperiamo dei lock e dei meccanismi wait/signal nel matchmaking. Questi ultimi fanno sì che il primo giocatore della partita 1 vs 1 entri in uno stato di attesa (wait()) senza sprecare cicli di CPU, finché un secondo giocatore non invia il segnale (set()) sbloccando la partita.

Progetto realizzato da:
Martina Esposito N39001953, Daniela Giannettino N39001974, Olimpia Manzo N39001952